



Rapport de Projet

Réalisé par :

Ayoub EL WAMY
ACHRAF KADDOURI
Abdellah OMALEK
Taha BENZAKI

Introduction

-Présentation générale du projet :

Le projet Todo List Manager est une application en ligne de commande développée en C++ qui permet à un utilisateur de gérer efficacement ses tâches quotidiennes. Il s'inscrit dans le cadre du module de Programmation Orientée Objet et vise à mettre en pratique les concepts fondamentaux de la POO tels que l'encapsulation, l'héritage, le polymorphisme et l'abstraction. L'application est organisée autour de plusieurs classes distinctes qui collaborent pour offrir une interface console claire et une persistance des données via des fichiers texte. Elle supporte les opérations CRUD (Create, Read, Update, Delete) sur des tâches, leur tri par priorité et la sauvegarde/chargement depuis le disque.

-Objectifs du projet :

- Appliquer les principes de la POO en C++ dans un contexte applicatif concret.
- Modéliser un système de gestion de tâches à l'aide de plusieurs classes interconnectées.
- Mettre en oeuvre la persistance de données via des fichiers texte (format CSV simplifié).
 - Explorer l'abstraction et le polymorphisme à travers une hiérarchie de notifications.
- Produire un code propre, modulaire et maintenable séparant les fichiers .h et .cpp.

-Problématique résolue :

Sans outil de gestion, les listes de tâches sont difficiles à maintenir et à prioriser. Ce projet résout cette problématique en proposant un gestionnaire interactif qui classe automatiquement les tâches par niveau d'urgence, notifie l'utilisateur du type de tâche ajoutée, et conserve les données entre les sessions grâce à la persistance fichier. La problématique centrale est donc : Comment structurer un système de gestion de tâches en C++ en respectant les principes de la POO, tout en assurant une expérience utilisateur fluide en ligne de commande ?

Analyse et Conception

-Fonctionnement général du système :

Au démarrage, le programme instancie un objet TaskManager qui maintient en mémoire une liste de tâches sous forme d'un vector<Task>. Un menu principal en boucle propose huit actions à l'utilisateur :

Option	Action
1	Ajouter une tâche
2	Modifier une tâche existante
3	Supprimer une tâche par identifiant
4	Afficher toutes les tâches
5	Trier les tâches par priorité
6	Sauvegarder dans un fichier texte
7	Charger depuis un fichier texte
0	Quitter le programme

-Relations entre les classes :

Relation	Entre	Description
Héritage	Notification -> UrgentTask /NormalTask	UrgentTask et NormalTask héritent de la classe abstraite Notification
Composition	TaskManager -> Task	TaskManager possède et gère le cycle de vie des objets Task
Dépendance	FileManager ->Task	FileManager utilise Task pour la sérialisation /désérialisation
Dépendance	main() -> TaskManager / FileManager	La fonction main délègue toutes les opérations aux deux managers

Description détaillée des classes

-Classe Task :

Rôle : Modélise une tâche individuelle. C'est l'entité centrale du projet. Elle encapsule l'identifiant, le titre, la catégorie, la priorité et l'état d'une tâche.

Attribut	Type	Visibilité	Description
id	int	private	Identifiant unique de la tâche
title	string	private	Titre descriptif de la tâche
category	string	private	Catégorie (ex. : Travail, Perso)
priority	string	private	Niveau de priorité (Urgent, Haute, Normale, Basse)
status	string	private	État (A faire, En cours, Terminé, etc.)

Méthodes importantes : Le constructeur par défaut initialise l'id à 0. Le constructeur paramétré initialise tous les attributs. La méthode `display()` affiche la tâche en format tabulaire grâce à `setw()` et `left` de `<iomanip>`. Les accesseurs (getters) et mutateurs (setters) assurent l'encapsulation de chaque attribut.

-Classe TaskManager :

Rôle : Orchestrateur principal de la logique métier. Elle maintient la collection de tâches et implémente toutes les opérations CRUD ainsi que le tri.

Méthode	Description
addTask()	Saisit les informations, vérifie la non-duplication de l'ID, puis insère
modifyTask()	Recherche par ID et permet de modifier titre, catégorie, priorité, statut
deleteTask()	Supprime la tâche correspondant à l'ID fourni par l'utilisateur
displayTasks()	Affiche toutes les tâches dans un tableau formaté avec séparateurs
sortTasks()	Trie le vecteur par priorité via <code>std::sort</code> et une lambda avec <code>priorityRank()</code>
getTasks()	Retourne une référence au vecteur (utilisée par FileManager)

-Classe TaskManager :

Rôle : Gère la persistance des données. Elle offre deux méthodes statiques permettant de sauvegarder le vecteur de tâches dans un fichier texte CSV (data/tasks.txt) et de le recharger en mémoire.

saveTasks() ouvre le fichier en écriture, crée le répertoire data/ si nécessaire (cross-plateforme via la macro MAKE_DIR), et sérialise chaque tâche sur une ligne avec le séparateur point-virgule.

loadTasks() ouvre le fichier en lecture, utilise istream pour découper chaque ligne par le caractère ';', reconstruit chaque objet Task et l'ajoute au vecteur. Une gestion d'exception protège contre les lignes malformées.

-Classe abstraite Notification et ses sous-classes :

Rôle : Introduit le polymorphisme dans le projet. Notification est une classe abstraite (interface) déclarant la méthode virtuelle pure alert(). Deux classes concrètes en héritent :

- **UrgentTask** : affiche un message d'alerte rouge [!] pour les tâches urgentes.
- **NormalTask** : affiche un message neutre [~] pour les tâches classiques.

Le destructeur virtuel dans Notification garantit la libération correcte de la mémoire lors de la destruction polymorphique des objets dérivés.

Choix Techniques

-Utilisation de `std::vector` :

Le conteneur `std::vector<Task>` a été préféré aux tableaux statiques car il offre une taille dynamique adaptée à un nombre variable de tâches. Il permet également d'utiliser les algorithmes standards de la STL comme `std::sort` et les itérateurs, tout en garantissant une gestion automatique de la mémoire heap.

-Persistance par fichiers texte :

Plutôt qu'une base de données (sqlite, etc.), le projet opte pour des fichiers texte au format CSV-like séparé par des points-virgules. Ce choix simplifie le déploiement (aucune dépendance externe), facilite la lecture humaine du fichier de données, et reste cohérent avec les objectifs pédagogiques du module. Les flux `ifstream` / `ofstream` combinés à `istreamstream` assurent une lecture ligne par ligne robuste.

-Organisation en fichiers `.h` / `.cpp` :

Chaque classe est séparée en un fichier d'en-tête (`.h`) contenant la déclaration et un fichier source (`.cpp`) contenant l'implémentation. Les gardes d'inclusion (`#ifndef` / `#define` / `#endif`) évitent les inclusions multiples.

La classe `Notification`, étant entièrement définie dans l'en-tête (méthodes inline courtes), n'a pas besoin de fichier `.cpp` séparé.

-Compatibilité multi-plateformes :

`FileManager` utilise une macro préprocesseur pour créer le répertoire `data/` de façon portable : `_mkdir` sur Windows et `mkdir` sur Linux/macOS. Cette approche assure que le projet compile et s'exécute correctement sur les deux environnements sans modification manuelle.

Exemple d'utilisation simulé

```
+-----+
| TASK MANAGER |
+-----+
| 1. Ajouter une tache |
| 4. Afficher les taches |
| 5. Trier les taches |
+-----+
```

Choix : 1
ID : 101
Titre : Preparer examen POO
Categorie: Etudes
Priorite :
1. Urgent 2. Haute 3. Normale 4. Basse
Choix : 1
Statut :
1. A faire 2. En cours ...
Choix : 2
[+] Tache ajoutee avec succes.
Choix : 4

ID	Titre	Categorie	Priorite	Statut
101	Preparer examen POO	Etudes	Urgent	En cours

1 tache(s) affichee(s).

Diagramme de Classes UML

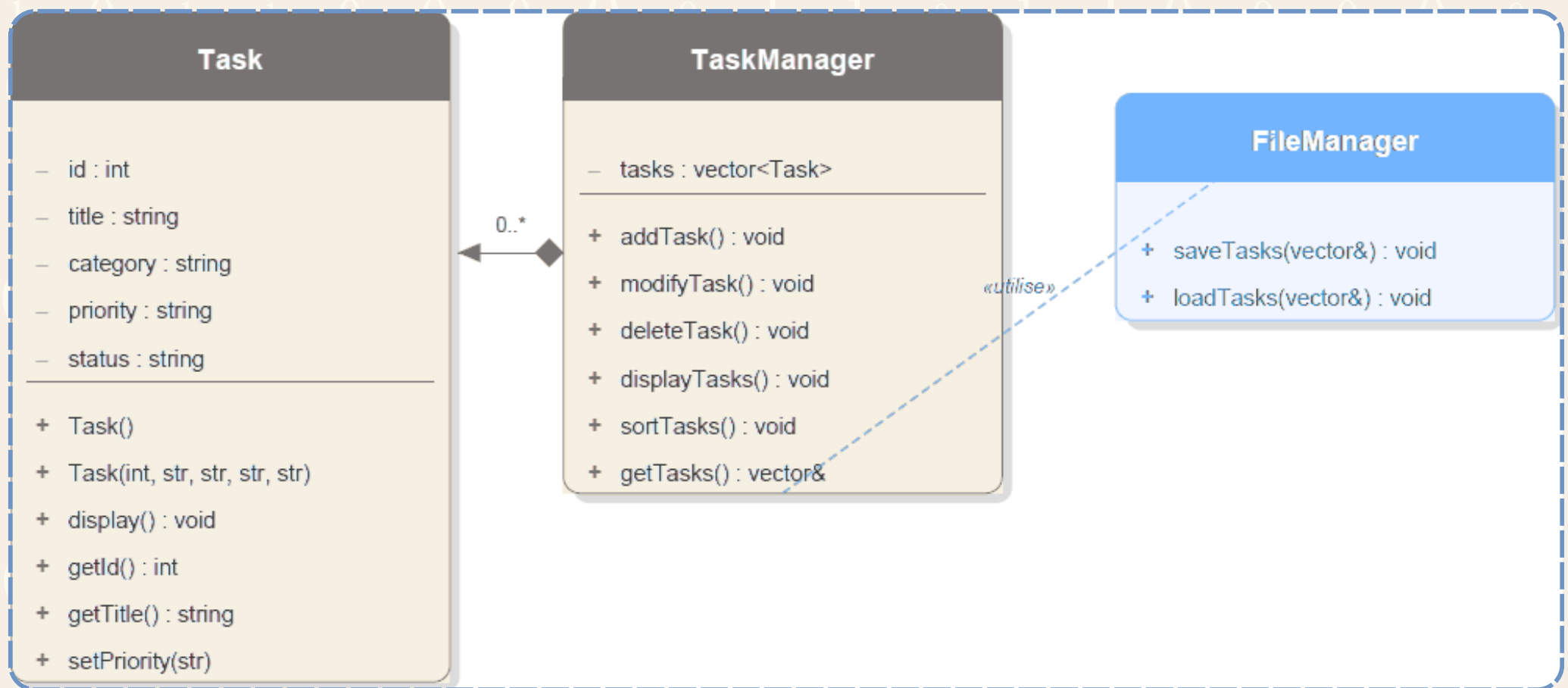
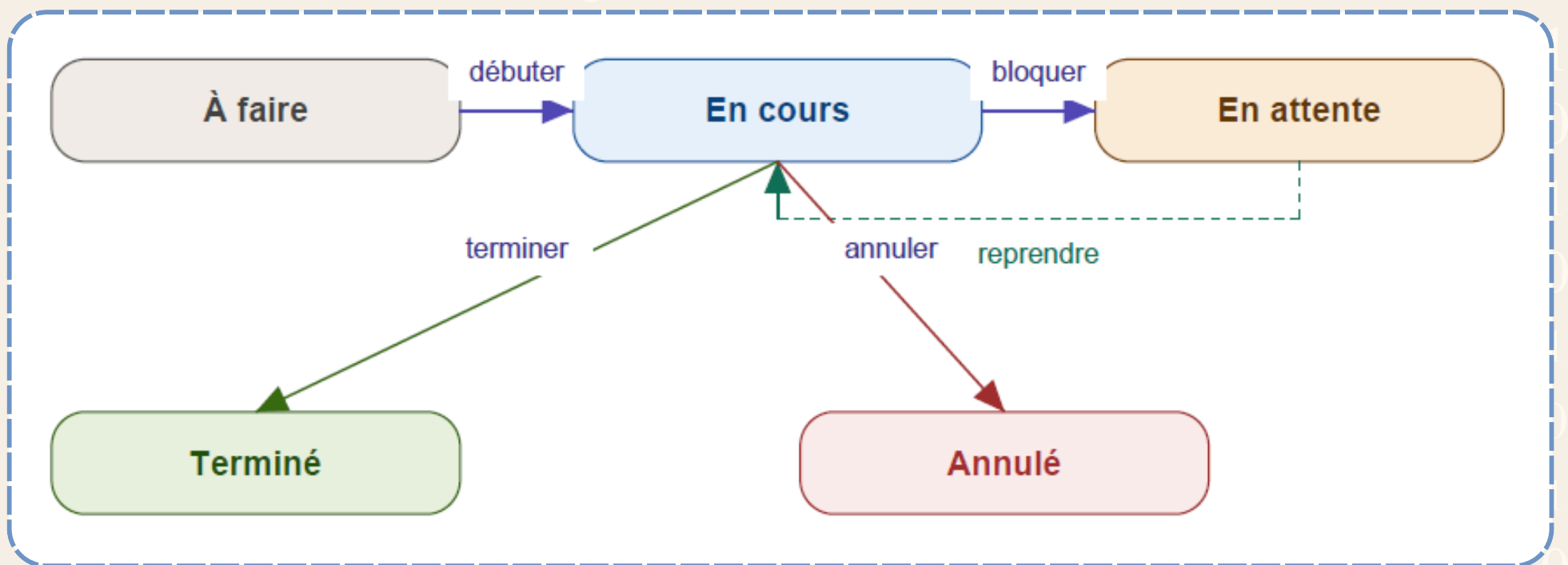


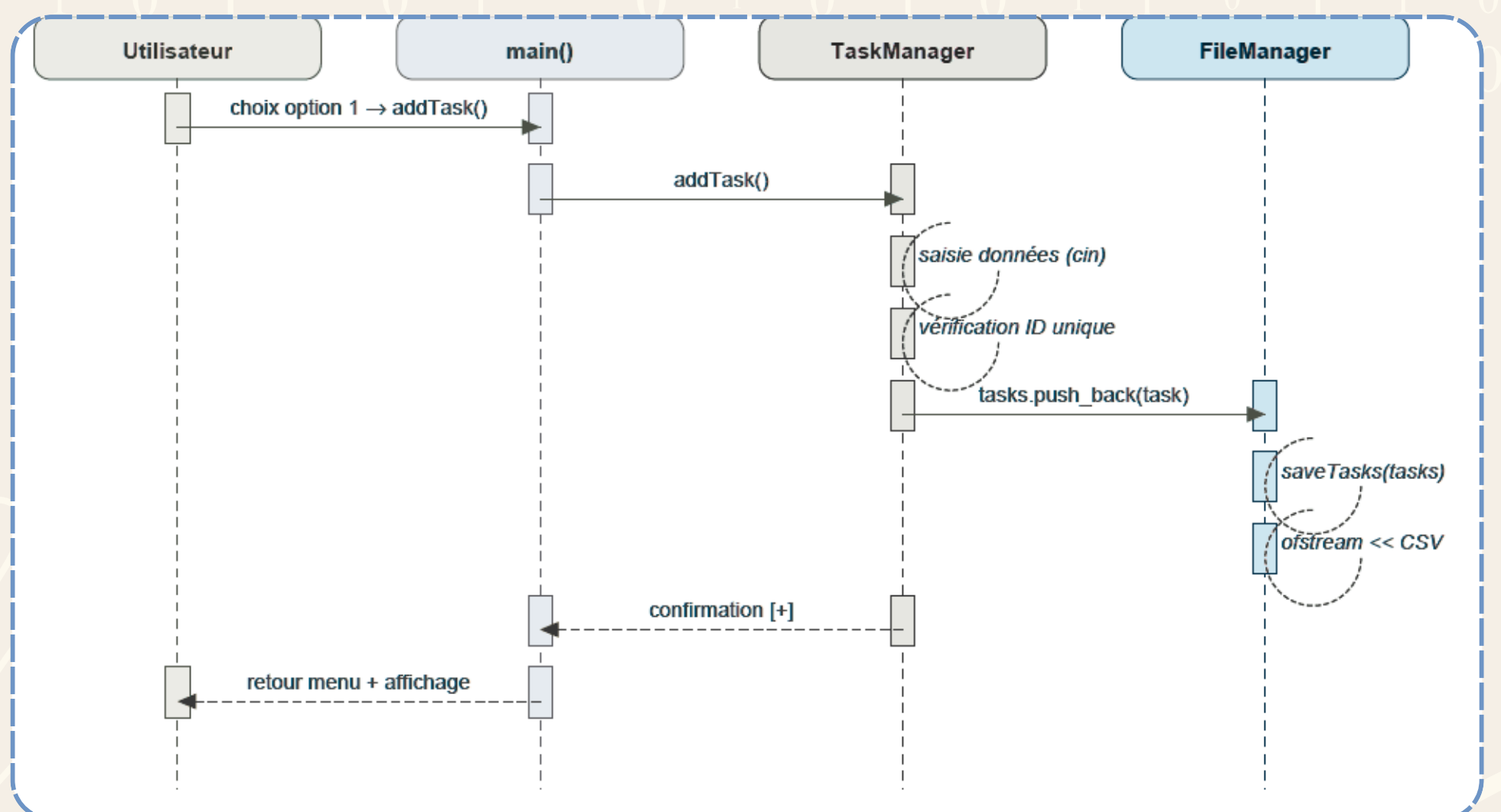
DIAGRAMME DE CAS D'UTILISATION



CYCLE DE VIE D'UNE TÂCHE



AJOUT D'UNE TÂCHE



Résumé du projet

Le projet Todo List Manager constitue une application C++ complète illustrant les principaux piliers de la Programmation Orientée Objet. La classe Task encapsule les données d'une tâche, TaskManager orchestre la logique métier, FileManager assure la persistance, et la hiérarchie Notification démontre l'héritage et le polymorphisme. La séparation .h / .cpp garantit une architecture modulaire et maintenable.



*Merci de votre
attention.*